



MeteoraAg DLMM

Security Assessment

March 25th, 2026 — Prepared by OtterSec

Xiang Yin

soreatu@osec.io

Gabriel Ottoboni

ottoboni@osec.io

Kevin Chow

kchow@osec.io

Julian Konchunas

julian@osec.io

Robert Chen

r@osec.io

Table of Contents

Executive Summary	3
Overview	3
Key Findings	3
Scope	4
Findings	5
Vulnerabilities	6
OS-MDM-ADV-00 Bin Manipulation Resulting in Swap Failure	8
OS-MDM-ADV-01 Failure to Include Transfer Fee in Exact-In Swap Amount	9
OS-MDM-ADV-02 Stale Bitmap After Limit-Order Swaps	10
OS-MDM-ADV-03 Liquidity Share Manipulation Risk	11
OS-MDM-ADV-04 Failure to Utilize Ceiling Division	12
OS-MDM-ADV-05 Possibility of Setting Excessive Transfer Fee	13
OS-MDM-ADV-06 Host Fee Underpayment	14
OS-MDM-ADV-07 Weight2 Transfer Fee Overcharge	16
OS-MDM-ADV-08 OnlyY Fee Mode Bypass	17
General Findings	19
OS-MDM-SUG-00 Unfair Reward Allocation	20
OS-MDM-SUG-01 Code Maturity	21
OS-MDM-SUG-02 Improving Validation Logic	22
OS-MDM-SUG-03 Code Refactoring	23
OS-MDM-SUG-04 Code Consistency	25

Appendices

Vulnerability Rating Scale	27
Procedure	28

01 — Executive Summary

Overview

MeteoraAg engaged OtterSec to assess the `dLmm` program. This assessment was conducted between January 28th and February 18th, 2025 and a follow-up review was conducted between February 9th and March 10th, 2026. For more information on our auditing methodology, refer to [Appendix B](#).

Key Findings

We produced 14 findings throughout this audit engagement.

In particular, we identified a high-risk vulnerability with the potential to exploit `go_to_a_bin` by moving the pool out of range and depositing minimal liquidity in intermediate bins, resulting in swaps aborting due to computational costs ([OS-MDM-ADV-00](#)). Also, in the exact-in swap path, the swap logic excludes the transfer-fee from the user's input transfer amount. As a result, when the input token charges transfer fees, the reserve receives less than the swap logic assumed ([OS-MDM-ADV-01](#)).

Additionally, excess composition fees are added to liquidity shares due to rounding, allowing manipulation of share values and resulting in a DoS or loss of funds when users deposit more than the worth of one share ([OS-MDM-ADV-03](#)). Furthermore, it is possible for anyone to call the protocol fee withdrawal function and specify the withdrawal amount, potentially forcing the protocol to incur unnecessary transfer fees when dealing with Token22 assets ([OS-MDM-ADV-06](#)).

We also suggested distributing rewards proportionally based on the liquidity amount in each bin rather than equally per bin ([OS-MDM-SUG-00](#)), and made recommendations to ensure adherence to coding best practices ([OS-MDM-SUG-01](#)).

We further advised incorporating extra checks within the codebase to enhance robustness ([OS-MDM-SUG-02](#)), and modifying the codebase for improved clarity, functionality, and mitigate potential security issues ([OS-MDM-SUG-03](#)).

Lastly, we highlighted minor consistency improvements, including aligning the API style for symmetric bin-pricing helpers, relaxing the rent-receiver to better reflect its passive role, utilizing the existing limit-order capability flag instead of a hardcoded value, and enforcing the same limit when increasing a position ([OS-MDM-SUG-04](#)).

02 — Scope

The source code was delivered to us in a Git repository at <https://github.com/MeteoraAg/DLMM>. This audit was performed against commit [8fbf3f2](#). Follow-up reviews were performed for [PR#353](#), [PR#354](#), [PR#355](#), [PR#360](#), [PR#411](#), and [c83256f](#).

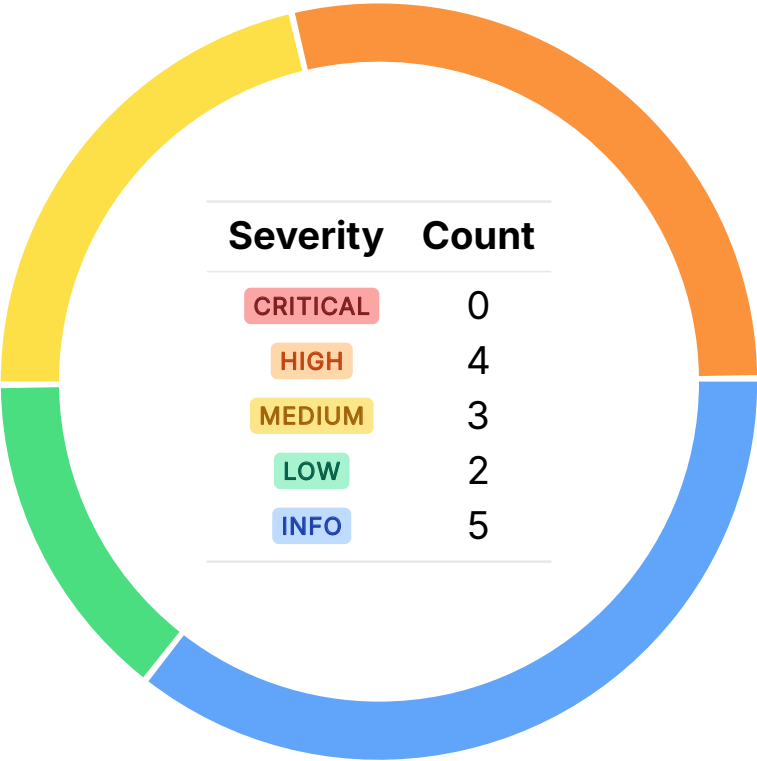
A brief description of the program is as follows:

Name	Description
dlmm	The liquidity book CLMM is a decentralized liquidity management and swapping platform that offers users high flexibility in providing liquidity, earning rewards, and performing swaps. It utilizes a bin-based approach, where liquidity is distributed across different bins, each representing a price range. This allows for concentrated liquidity provisioning, thereby enhancing capital efficiency for liquidity providers.

03 — Findings

Overall, we reported 14 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in [Appendix A](#).

ID	Severity	Status	Description
OS-MDM-ADV-00	HIGH	RESOLVED ✓	It is possible to exploit <code>go_to_a_bin</code> by moving the pool out of range and depositing minimal liquidity in intermediate bins, causing swaps to abort due to computational costs.
OS-MDM-ADV-01	HIGH	RESOLVED ✓	In the exact-in swap path, <code>perform_swap</code> utilizes the transfer-fee-excluded <code>user_pay_amount</code> as the user's input transfer amount. As a result, when the input token charges transfer fees, the reserve receives less than the swap logic assumed.
OS-MDM-ADV-02	HIGH	RESOLVED ✓	Swaps may change a bin array's effective emptiness by moving limit-order state between <code>open_order_amount</code> and <code>processed_order_remaining_amount</code> , but <code>BinArrayBitmapExtension</code> is not updated accordingly. This may leave the bitmap stale and arrays with not-yet-cashed-out limit-order liquidity will be skipped.
OS-MDM-ADV-03	HIGH	RESOLVED ✓	Excess composition fees are added to liquidity shares due to rounding, allowing for manipulation of share values, resulting in a denial-of-service or loss of funds when users deposit more than the worth of one share.
OS-MDM-ADV-04	MEDIUM	RESOLVED ✓	<code>calculate_pre_fee_amount</code> improperly derives the <code>pre_fee_amount</code> , resulting in an incorrect transfer fee calculation.

OS-MDM-ADV-05	LOW	RESOLVED ✓	Anyone may call <code>withdraw_protocol_fee</code> and specify the withdrawal amount, potentially forcing the protocol to incur unnecessary transfer fees when dealing with Token22 assets.
OS-MDM-ADV-06	LOW	RESOLVED ✓	The host fee is deducted separately before the main swap, but the transfer fee is calculated only once on the total amount. This results in the host receiving slightly less than intended due to rounding discrepancies.
OS-MDM-ADV-07	MEDIUM	RESOLVED ✓	<code>add_liquidity_by_weight2</code> does not preserve the user's specified amounts as hard max spend limits when transfer fees apply. As a result, <code>handle_deposit_by_amounts2</code> can pull more tokens than the user expected after converting the net deposit totals into transfer-fee-included amounts.
OS-MDM-ADV-08	MEDIUM	RESOLVED ✓	In <code>OnlyY</code> pools, single-sided active-bin deposit and withdrawal by removing liquidity may still route composition fees onto token <code>X</code> , even though swap-like actions are supposed to charge fees only in <code>Y</code> . This enables an attacker to emulate a <code>X→Y</code> swap path and bypass the intended fee-token policy.

Bin Manipulation Resulting in Swap Failure HIGH

OS-MDM-ADV-00

Description

It is possible to leverage the `go_to_a_bin` instruction to manipulate the active liquidity bin, disrupting normal swap operations or exploiting rounding mechanics to gain an unfair advantage. Utilizing `go_to_a_bin`, it is possible to change the pool's `lb.active_id` to a bin far from its current position. Since there is an optimization that skips entirely empty bin arrays, the attacker must deposit some liquidity to ensure every bin is checked during the swap.

Consequently, when a user attempts to swap tokens, `internal_handle_swap` iterates over the liquidity bins from the `active_id`. Since each intermediate bin now contains some liquidity, the swap process must traverse through all these bins. This dramatically increases the CU cost because each bin must be checked, causing the swap to be aborted.

Remediation

Set a minimum liquidity deposit requirement per bin to prevent attackers from utilizing atomic units to populate bins, rendering the attack too expensive to perform.

Patch

Resolved in [43b3a79](#).

Failure to Include Transfer Fee in Exact-In Swap Amount HIGH OS-MDM-ADV-01

Description

In the exact-in path, `internal_handle_swap2` passes `swap_final_result.user_pay_amount` into `perform_swap`, but that value is the transfer-fee-excluded net input utilized by the swap math, not the gross amount the user must send. If the input token charges a transfer fee, `transfer_from_user2` deducts that fee during transfer, so the reserve receives less than the amount assumed by `process_swap`. Before the refactor, this path passed the original `amount_in`, which already included token transfer fees from the user's perspective. Thus, the refactor regressed this by switching to `swap_final_result.user_pay_amount` without including transfer fees.

```
>_ lb_clmm/src/instructions/v2/swap2.rs
```

RUST

```
fn internal_handle_swap2<'c: 'info, 'info>(
    ctx: &Context<'_, '_, 'c, 'info, Swap2<'info>>,
    amount_in: u64,
    exact_amount_out: Option<u64>,
    remaining_accounts_info: RemainingAccountsInfo,
) -> Result<InternalSwapResult> {
    [...]
    if exact_amount_out.is_none() {
        // swap exact in
        ctx.accounts.perform_swap(
            swap_for_y,
            fee_mode.fees_on_input,
            swap_final_result.host_fee,
            swap_final_result.user_pay_amount,
            swap_final_result.user_receive_amount,
            transfer_hook_in_accounts,
            transfer_hook_out_accounts,
        )?;
    }
    [...]
}
```

Remediation

Convert `user_pay_amount` back to a transfer-fee-included gross amount, as already done in the exact-out branch, before calling `perform_swap`.

Patch

Resolved in [7673dcc](#).

Stale Bitmap After Limit-Order Swaps HIGH

OS-MDM-ADV-02

Description

In the swap path, limit-order state may move from `open_order_amount` into `processed_order_remaining_amount`, so a bin array may remain logically non-empty even though its open-order field was cleared. If the swap instruction does not flip zero-liquidity flags into `BinArrayBitmapExtension`, the bitmap may become stale and fail to mark arrays that still contain not-yet-cashed-out limit-order liquidity. This breaks consistency between the actual bin contents and the pair's global liquidity index utilized to locate active bin arrays, thereby hindering future bin discovery.

> `lb_clmm/src/state/bin.rs`

RUST

```
pub fn get_zero_liquidity_flags(&self, is_support_limit_order: bool) -> Vec<bool> {
    let mut flags = vec![];
    for bin_array in self.bin_arrays.iter() {
        flags.push(bin_array.is_zero_liquidity(is_support_limit_order));
    }
    flags
}
```

Remediation

Apply bitmap flip logic during swap (`get_zero_liquidity_flags` should be flipped in swap instruction) and make `bin_array_bitmap_extension` mutable in `swap2` so those state transitions may be persisted.

Patch

Resolved in [30cc532](#).

Liquidity Share Manipulation Risk HIGH

OS-MDM-ADV-03

Description

`add_liquidity` may result in rounding issues when calculating liquidity shares for users. Excess composition fees created by rounding errors are added to liquidity shares, rendering them more valuable. When liquidity shares are small, this may be exploited by repeatedly calling `remove_liquidity` to withdraw 99.99% of the liquidity. Each withdrawal reduces the liquidity pool, but the remaining shares become exponentially more valuable due to redistributed rounding errors. This may result in a denial-of-service or loss of funds if users deposit more than the value of one liquidity share.

Remediation

Implement a check for the minimum liquidity share or return the excess rounding back to the user.

Patch

Resolved in [43b3a79](#).

Failure to Utilize Ceiling Division MEDIUM

OS-MDM-ADV-04

Description

`token2022::calculate_pre_fee_amount` should perform a ceiling division operation in the calculation of the `raw_pre_fee_amount`. Currently, it replaces the denominator of the division calculation with `ONE_IN_BASIS_POINTS`. The original intent of the calculation is to scale the numerator according to the fee basis points, ensuring that the result is rounded up appropriately. However, replacing the denominator with `ONE_IN_BASIS_POINTS` incorrectly scales the numerator by a large factor. As a result, transfer fee calculation becomes incorrect.

```
>_ src/utills/token2022.rs RUST

pub fn calculate_pre_fee_amount(transfer_fee: &TransferFee, post_fee_amount: u64) -> Option<u64>
    → {
    [...]
    else {
        let numerator = (post_fee_amount as u128).checked_mul(ONE_IN_BASIS_POINTS)?;
        let denominator = ONE_IN_BASIS_POINTS.checked_sub(transfer_fee_basis_points)?;
        // let raw_pre_fee_amount = ceil_div(numerator, denominator)?;
        let raw_pre_fee_amount = numerator
            .checked_add(ONE_IN_BASIS_POINTS)?
            .checked_sub(1)?
            .checked_div(denominator)?;

        if raw_pre_fee_amount.checked_sub(post_fee_amount as u128)? >= maximum_fee as u128 {
            post_fee_amount.checked_add(maximum_fee)
        }[...]
    }
}
```

Remediation

Update the calculation to utilize

`ceil_div ( ceil_div((numerator + denominator - 1) / denominator) ).`

Patch

Resolved in [d58175d](#).

Possibility of Setting Excessive Transfer Fee LOW

OS-MDM-ADV-05

Description

It is possible for anyone to call `withdraw_protocol_fee` and specify the withdrawal amounts on behalf of the fee owner. While the instruction correctly validates that the requested withdrawal amounts do not exceed the accumulated protocol fees, it does not enforce an optimal withdrawal strategy to minimize transfer fees when dealing with Token-2022 (T22) tokens that have built-in transfer fees. Thus, if the pool contains a T22 with transfer fees, this may be utilized to make the protocol pay more in transfer fees than necessary.

Remediation

Only allow the fee owner to call `withdraw_protocol_fee`, preventing external actors from choosing arbitrary amounts.

Patch

Resolved in [8e0f9ce](#).

Host Fee Underpayment LOW

OS-MDM-ADV-06

Description

In `swap2`, when a user initiates a swap, a transfer fee is subtracted from the specified `amount_in` (amount of tokens given by the user for the swap). This subtracted amount is what is actually utilized for the swap. This transfer fee is calculated as if it was a single transfer. The issue arises when a host fee is specified (`host_fee_in`). In this case, `perform_swap` performs two separate transfers. First, the host fee is deducted from the user and sent to the host, and in the second transfer, the remaining amount is transferred from the user to the liquidity pool.

```
>_ src/utls/token2022.rs RUST

fn internal_handle_swap2<'c: 'info, 'info>(
    ctx: &Context<'_, '_, 'c, 'info, Swap2<'info>>,
    amount_in: u64,
    remaining_accounts_info: RemainingAccountsInfo,
) -> Result<InternalSwapResult> {
    [...]
    let transfer_fee_excluded_amount_in =
        transfer_fee_excluded_amount_in.safe_sub(total_host_fee)?;

    // very edge case that calculate_transfer_fee_included_amount return result greater than
    // ↳ amount_in
    // due to function calculate_inverse_fee is round up
    let transfer_fee_included_amount_in =
        calculate_transfer_fee_included_amount(token_mint_in, transfer_fee_excluded_amount_in)?
            .amount
            .min(amount_in);

    let transfer_fee_included_host_fee = amount_in.safe_sub(transfer_fee_included_amount_in)?;
    [...]
```

However, the calculation in `calculate_transfer_fee_excluded_amount` assumes that only one transfer occurs, so the entire `amount_in` is adjusted for fees before transferring. However, in reality, the host fee is deducted in a separate transaction. Since each transfer is subject to a fee, the host actually receives less than the `total_host_fee` they were supposed to get.

Remediation

Modify `calculate_transfer_fee_excluded_amount` to account for the fact that the host fee is deducted separately. Instead of assuming a single transfer, explicitly compute the transfer fee twice—once on the host fee portion and once on the swap transfer amount. This ensures the host receives the full intended amount after all transfer fees are properly accounted for.

Patch

Resolved in [d58175d](#).

Weight2 Transfer Fee Overcharge MEDIUM

OS-MDM-ADV-07

Description

The new `add_liquidity_by_weight2` path is inconsistent with the other Token-2022 add-liquidity flows because it forwards its computed per-bin allocation directly into `handle_deposit_by_amounts2`, which later converts the net deposit totals into transfer-fee-included gross transfer amounts. Unlike the other Token-2022 liquidity instructions, it does not first treat the user-specified amounts as capped max inputs or verify that the final gross debit stays within those bounds.

Remediation

Align `add_liquidity_by_weight2` with the other v2 deposit paths by first interpreting the user-provided amounts as transfer-fee-inclusive maximums, converting them to transfer-fee-excluded effective budgets before computing `amounts_in_bin`. Also, add an explicit post-computation check that the final transfer-fee-included totals do not exceed the caller's declared maxima.

Patch

Resolved in [2503e37](#).

OnlyY Fee Mode Bypass

MEDIUM

OS-MDM-ADV-08

Description

In pools configured with `fee_mode = OnlyY`, swap-like operations are expected to charge fees exclusively in token `Y`. However, `compute_composition_fee` determines the fee side from the deposit's inferred synthetic swap direction and `FeeMode::get_fee_mode`. without enforcing the higher-level expectation that composition fees in an `OnlyY` pool must never be collected in token `X` under single-sided deposit flows.

> _ lb_clmm/src/instructions/deposit/add_liquidity.rs

RUST

```
pub fn get_deposit_bin_result(
    amount_x_into_bin: u64,
    amount_y_into_bin: u64,
    in_id: i32,
    bin: &Bin,
    lb_pair: &LbPair,
) -> Result<DepositBinResult> {
    [...]
    let CompositionFeeResult {
        composition_fee_x,
        composition_fee_y,
        protocol_fee_x,
        protocol_fee_y,
        fees_on_input,
    } = compute_composition_fee(
        out_amount_x,
        amount_x_into_bin,
        out_amount_y,
        amount_y_into_bin,
        lb_pair,
    );
    [...]
```

This becomes exploitable through `add_liquidity_single_side` on token `X` into the active bin. Since this is similar to a swap, the deposit path may charge a composition fee. Because that fee computation may still resolve to token `X`, the user effectively pays the composition fee in `X` even though the pool's configured fee policy is supposed to restrict fees to `Y`. The deposit may be followed with a call to `remove_liquidity`, withdrawing the newly minted liquidity share. By repeating this cycle utilizing the returned token `X` as the next input, the attacker may iteratively transform `X` into `Y` while continuing to pay most of the fee in `X` rather than `Y`, bypassing the pool's configured `OnlyY` fee policy.

Remediation

Enforce fee-token selection consistently for composition fees, so `onlyY` pools are restricted from charging active-bin composition fees in token `X`.

Patch

Resolved in [c83256f](#).

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-MDM-SUG-00	Rewards are distributed equally across bins crossed, allowing minimal liquidity to disproportionately claim rewards.
OS-MDM-SUG-01	Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices.
OS-MDM-SUG-02	There are several instances where proper validation should be performed to prevent potential security issues.
OS-MDM-SUG-03	The codebase may be updated for improved clarity, functionality, and mitigate potential security issues.
OS-MDM-SUG-04	There are minor consistency improvements, including aligning the API style for symmetric bin-pricing helpers, relaxing the rent-receiver to better reflect its passive role, utilizing the existing limit-order capability flag instead of a hardcoded value, and enforcing the same limit when increasing a position.

Unfair Reward Allocation

OS-MDM-SUG-00

Description

Rewards are distributed equally across all bins a position spans, regardless of the liquidity size in each bin. This approach introduces a potential vulnerability, particularly when extremely small (or dust) liquidity deposits are involved. It enables a bin containing just one atomic unit of liquidity to receive the same reward as a bin with a much larger liquidity amount. Consequently, a bin with a minimal amount of liquidity (dust) earns disproportionately large rewards.

Remediation

Distribute rewards proportionally based on the liquidity amount in each bin rather than equally per bin.

Code Maturity

OS-MDM-SUG-01

Description

1. There is a minor inconsistency in how bin IDs are checked in `advance_active_bin` and `InitializeBinArray`, as they utilize global constants (`MIN_BIN_ID`, `MAX_BIN_ID`) to validate bin IDs, rather than utilizing the pair-specific fields (`min_bin_id` and `max_bin_id`). It is always more appropriate to check against the pair's fields.
2. In `token2022`, the check for `transfer_fee_basis_points == 100%` in `calculate_transfer_fee_included_amount` is redundant, as `calculate_pre_fee_amount` will handle this case correctly later in the process.

```
>_ lb_clmm/src/utils/token2022.rs RUST

pub fn calculate_transfer_fee_included_amount<'info>(
    token_mint: &InterfaceAccount<'info, Mint>,
    transfer_fee_excluded_amount: u64,
) -> Result<TransferFeeIncludedAmount> {
    [...]
    if u16::from(epoch_transfer_fee.transfer_fee_basis_points) == MAX_FEE_BASIS_POINTS
        ↪ {
        // edge-case: if transfer fee rate is 100%, current SPL implementation returns
        ↪ 0 as inverse fee.
        // https://github.com/solana-labs/solana-program-library/blob/
        // fe1ac9a2c4e5d85962b78c3fc6aaf028461e9026/token/program-2022/src/
        // extension/transfer_fee/mod.rs#L95
        // But even if transfer fee is 100%, we can use maximum_fee as transfer fee.
        // if transfer_fee_excluded_amount + maximum_fee > u64 max, the
        ↪ followingchecked_add should fail.
        u64::from(epoch_transfer_fee.maximum_fee)
    }
    [...]
}
```

3. Validate `bin_step > 0` when initializing `PresetParameter` and `PermissionLbPair` to ensure a valid state and consistency with `CustomizablePermissionlessLbPair`.

Remediation

Implement the above-mentioned suggestions.

Improving Validation Logic

OS-MDM-SUG-02

Description

1. Setting a reasonable upper limit on `increase_oracle_length` is recommended to prevent performance issues created by unlimited reallocation, which may unnecessarily increase account sizes.
2. The rebalancing logic currently lacks a bounds check to ensure a position's bin range stays within the allowed `min_bin_id` and `max_bin_id`. Without this, it may be possible to create `BinArrays` outside these bounds, rebalance a position and deposit out of bounds, and maybe even swap the `active_bin` out of bounds even though this is unlikely. Add a bin bounds check to rebalance, similar to what is implemented in `increase_position_length`.

```
>_ lb_clmm/src/instructions/position/increase_position_length.rs RUST

pub fn handle(ctx: Context<IncreasePositionLength>, length_to_add: u16, side: u8) ->
    ↳ Result<()> {
    [...]
    require!(
        lower_bin_id >= lb_pair.parameters.min_bin_id
        && upper_bin_id <= lb_pair.parameters.max_bin_id,
        LBEError::InvalidBinId
    );
    [...]
}
```

3. Utilize checked casts in `add_liquidity_by_strategy`, specifically for calculations within `to_weight_descending_order` and `to_weight_curve`, in order to improve safety and prevent potential overflow or underflow issues.

Remediation

Add the missing validations in the codebase.

Code Refactoring

OS-MDM-SUG-03

Description

1. The property test for limit orders only generates `ask-side` limit orders because `rng.gen_range(0, 1)` always returns zero, so bid-side flows are never exercised. While, utilizing `rng.gen()` fixes this, it exposes a second issue in `place_limit_order_and_assert`, where the helper checks `before_pending_amount > 0` in a branch that is only reached when pending amount is already zero. To match the contract logic, that condition should instead utilize `before_bin_data.total_processing_order_amount > 0`.

```
> _ lb_clmm/src/tests/limit_order_tests/limit_order_common_functions.rs RUST

pub fn place_limit_order_and_assert([...]) {
    [...]
    else {
        if is_ask_side != before_ask_side && before_pending_amount > 0 {
            assert_eq!(bin.total_processing_order_amount, 0);
            assert_eq!(
                bin.order_age,
                before_bin_data.order_age.checked_add(1).unwrap()
            );
        } [...]
    } [...]
}

proptest! {
    #[test]
    fn test_limit_order_flow_random([...]) {
        [...]
        match rng.gen_range(0, 7) {
            [...]
            3 => {
                let is_ask_side = rng.gen_range(0, 1) == 0;
                let amount = rng.gen_range(0, place_limit_order_amount);
                users[user_index].place_limit_order(&mut pool, amount, is_ask_side);
            }
            [...]
        }
    }
}
```

2. Utilize `swap_for_y != is_ask` in `Bin::get_current_limit_order_amounts` as it is easier to understand than `swap_for_y != is_ask` because it directly expresses that the branch is taken when the two booleans differ. For boolean values, `!=` and `XOR` are equivalent, so this change does not alter behavior and improves readability.


```

>_ lb_clmm/src/state/bin.rs RUST

fn get_current_limit_order_amounts(&self, swap_for_y: bool) -> Result<(u64, u64)> {
    // let limit_order_side = self.get_limit_order_side()?;
    let pending_limit_order_amount = self.get_pending_limit_order_amount()?;
    if pending_limit_order_amount == 0 {
        Ok((0, 0))
    } else {
        let is_ask = self.is_limit_order_ask_side();
        // swap_for_y && !is_ask || !swap_for_y && is_ask is equal to swap_for_y XOR
        ⇔ is_ask
        if swap_for_y ^ is_ask {
            Ok((
                self.open_order_amount,
                self.processed_order_remaining_amount,
            ))
        } else {
            Ok((0, 0))
        }
    }
}

```

- After `get_or_store_bin_price` was removed, bins now rely on the stored price field being initialized. Consequently, on scanning the mainnet for legacy bins with an unset `price` field, some such bins were identified, but only within out-of-bounds bin arrays with no liquidity, and therefore do not appear reachable through normal active pool flows. Nonetheless, they should be closed via `close_bin_array` to ensure proper cleanup and reduce future edge-case risk.

Remediation

Incorporate the above-stated refactors.

Code Consistency

OS-MDM-SUG-04

Description

1. `Bin::calculate_filled_amount_exact_in` utilizes `self.get_amount_in` for one price conversion but `Bin::get_amount_out` for the opposite conversion, even though both are conceptually symmetric bin-pricing helpers. Ensure both helpers utilize the same API style for consistency.

```
>_ lb_clmm/src/state/bin.rs RUST

fn calculate_filled_amount_exact_in(
    &self,
    amount_in: u64,
    total_out_amount: u64,
    swap_for_y: bool,
) -> Result<SwapAmountFromInput> {
    let total_amount_in = self.get_amount_in(total_out_amount, swap_for_y, Rounding::Up)?;
    if amount_in >= total_amount_in {
        Ok(SwapAmountFromInput {
            amount_in: total_amount_in,
            amount_left: amount_in.safe_sub(total_amount_in)?,
            out_amount: total_out_amount,
        })
    } else {
        let out_amount = Bin::get_amount_out(amount_in, self.price, swap_for_y)?;
        [...]
    }
}
```

2. `CloseBinArray::rent_receiver` only receives refunded lamports through `close = rent_receiver` and does not authorize any state transition, so requiring it to be a `Signer<'info>` is unnecessarily restrictive. Utilize a mutable `UncheckedAccount<'info>` to make the instruction interface more consistent.

```
>_ lb_clmm/src/instructions/admin/close_bin_array.rs RUST

#[derive(Accounts)]
pub struct CloseBinArray<'info> {
    [...]

    #[account(mut)]
    pub rent_receiver: Signer<'info>,

    pub signer: Signer<'info>,
}
```

3. `swap2::process_swap` already computes `is_support_limit_order`, but the pre-swap emptiness snapshot still calls `active_bin_array.is_zero_liquidity(true)` with a hardcoded value. Pass `is_support_limit_order` instead to make the check match the actual pair configuration and keep it consistent with the rest of the function.
4. The enforcement of limits when increasing a position is inconsistent. Currently, `rebalance` and `increase_position_length2` utilize `MAX_PERMITTED_DATA_INCREASE`, whereas `increase_position_length` still utilizes `MAX_RESIZE_LENGTH`. Ensure that all related paths utilize `MAX_PERMITTED_DATA_INCREASE` for uniformity.

Remediation

Include the above suggestions.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the [General Findings](#).

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
 - Improperly designed economic incentives leading to loss of funds.
-

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
 - Exploitation involving high capital requirement with respect to payout.
-

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
 - Forced exceptions in the normal user flow.
-

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.
-

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
 - Improved input validation.
-

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.